Project No: A2285-251

TinyLlama Inference Acceleration on FPGA Using High-Level Synthesis

Submitted by: Yap Chen Kit

Supervisor: Professor Chang Chip Hong

Co-supervisor: Koca Nazim Altar

School of Electrical and Electronic Engineering

A final year project report presented to Nanyang Technological University
in partial fulfilment of the requirements for the
Degree of Bachelor of Engineering

2026

Contents

Abstract **iv**

Acknowledgement **v**

Acronyms **vi**

Symbols **vii**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**
1.1 Motivations 1
1.2 Objectives and Scope 2
1.3 Organisation of Report 2

2 Literature Review **3**
2.1 Transformer Model Architecture 3
2.2 TinyLlama Model Architecture 4
2.2.1 Token Embeddings 5
2.2.2 Rotary Positional Embedding 6
2.2.3 RMSNorm 7
2.2.4 Grouped-Query Self-Attention 8
2.2.5 SwiGLU Feed-Forward Network 9
2.3 FPGA Acceleration of Neural Networks 10
2.4 Quantisation for Efficient Inference 11
2.5 HLS-Based Hardware Design 11
2.5.1 Pipelining and the Initiation Interval 11
2.5.2 Scaling Transformations: Unrolling and Dataflow 12
2.5.3 Memory Transformations 12

3 System Architecture and Design **14**
3.1 Top-Level System Overview 14

3.2 Hardware/Software Partitioning 14
3.3 Transformer Layer Architecture 15
3.3.1 RMSNorm 15
3.3.2 Multi-Head Attention 16
3.3.3 SwiGLU Feed-Forward Network 16
3.4 Memory Architecture and Data Flow 16
3.5 Quantisation Scheme 16

4 Implementation 18
4.1 HLS Kernel Development Environment 18
4.2 Matrix–Vector Multiplication Kernel 18
4.3 Pragma Optimisation Strategy 19
4.3.1 Array Partitioning 19
4.3.2 Loop Unrolling 20
4.4 RMSNorm Implementation 20
4.5 Attention Kernel Implementation 20
4.6 Host Software and XRT Driver Layer 21
4.7 Vivado Integration and Bitstream Generation 21

5 Results and Discussion 22
5.1 Experimental Setup 22
5.2 HLS Synthesis Results 22
5.3 Functional Verification 23
5.4 Inference Performance 23
5.5 Energy Efficiency 23
5.6 Comparison Against Prior Work 24
5.7 Discussion 24

6 Conclusion and Future Work 25
6.1 Conclusion 25
6.2 Recommendations for Future Work 26

Reflection on Learning Outcome Attainment 27

Acknowledging/Declaring the Use of GAI 28

References 28

A HLS Kernel Source Code 29

B Vivado Synthesis and Implementation Reports **30**

C Quantisation Calibration Data **31**

Abstract

Small-scale large language models (LLMs) such as TinyLlama-1.1B have emerged as a compelling class of compact transformer models that deliver meaningful natural language understanding capabilities within a fraction of the memory footprint of frontier models such as GPT-4. Yet even at the billion-parameter scale, these models impose substantial computational demands that limit their deployment in embedded and edge environments.

Currently, these language models are mostly deployed on Graphics Procesing Units (GPU). However, GPU inference is energy-intensive and memory-bandwidth-bound, particularly during the autoregressive decode phase since the time and memory complexity of self-attention are quadratic in sequence length.

Field-programmable gate arrays (FPGAs) offer a compelling alternative to GPU-based inference by providing reconfigurable hardware with low power consumption and predictable latency. But mapping a billion-parameter transformer to resource-constrained FPGA fabric presents significant challenges in quantization, memory tiling, and hardware-software partitioning.

This project starts off with a comprehensive profiling of a RTX4060 Laptop GPU running a pytorch inference of the Llama 2-based TinyLlama 1.1B model from HuggingFace with default GPU kernel settings to observe for bottlenecks and to measure propotion of wall-clock time required by the self-attention and feed-forward layer during the forward pass. Based on the results, an FPGA-based accelerator is proposed to accelerate the architecture of the Tinyllama model. For this project, emphasis is placed on the hardware implementation and optimization of the self attention layer and the feed-forward layer.

This project implements and optimises the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 FPGA platform using Vitis High-Level Synthesis (HLS). The core transformer components – including RMSNorm, matrix–vector multiplication, multi-head attention, and the SwiGLU feed-forward network – were implemented as quantised INT8 HLS kernels.

(The results demonstrate that FPGA acceleration with INT8 quantisation achieves competitive inference performance while operating within the ZCU102's resource budget, validating the viability of transformer-based language model deployment on reconfigurable hardware.)

Acknowledgement

I would like to express my sincere gratitude to Professor Chang Chip Hong for being my supervisor and giving me multiple chances to explore different research areas since my second year until my final year of undergraduate studies, and allowing me to pursue this topic as my FYP.

I would also like to thank my co-supervisor, Koca Nazim Altar, for supporting me during my final year project, in addition to his full-time job and his PhD tasks. His guidance have helped me overcome technical and mental roadblocks with his kindness and expertise. Altar have provided me valuable knowledge that helped bring this project to fruition.

Yap Chen Kit
May 2026

Acronyms

AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
DDR	Double Data Rate (SDRAM)
DMA	Direct Memory Access
DSP	Digital Signal Processor
FIFO	First-In, First-Out
FPGA	Field-Programmable Gate Array
FF	Flip-Flop
GeLU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
HLS	High-Level Synthesis
II	Initiation Interval
INT8	8-bit Integer
IP	Intellectual Property (core)
LLM	Large Language Model
LUT	Look-Up Table
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
NPU	Neural Processing Unit
PL	Programmable Logic
PS	Processing System
PTQ	Post-Training Quantisation
QKV	Query, Key, Value
ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Layer Normalisation
RTL	Register-Transfer Level
SoC	System-on-Chip
SwiGLU	Swish-Gated Linear Unit
XRT	Xilinx Runtime
ZCU102	Zynq UltraScale+ MPSoC Evaluation Board (ZCU102)

Symbols

Symbol	*Description*
d_{model}	Hidden dimension of the model (2048 for TinyLlama-1.1B)
d_{ff}	Feed-forward network hidden dimension
d_k	Dimension of each attention head
L	Number of transformer layers
H	Number of attention heads
V	Vocabulary size
T	Sequence length (number of tokens)
W_Q, W_K, W_V	Query, Key, Value projection weight matrices
W_O	Output projection weight matrix
Q, K, V	Query, Key, Value activation matrices
softmax	Softmax normalisation function
s	Quantisation scale factor
z	Quantisation zero-point offset
b	Number of quantisation bits
$\hat{x}$	Quantised representation of variable x
II	Initiation interval of a pipelined loop (clock cycles)
f_{clk}	Clock frequency (MHz)
N_{LUT}	Number of look-up tables consumed
N_{DSP}	Number of DSP blocks consumed
P	Power consumption (mW)

List of Figures

2.1 Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N-1)$ for N inputs **PLACEHOLDER**. . 12

2.2 Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast **PLACEHOLDER**. 12

List of Tables

2.1 TinyLlama-1.1B architectural hyperparameters **PLACEHOLDER**. . 4

5.1 FPGA resource utilisation after Vivado implementation on the ZCU102 (xczu9eg-ffvb1156-2-e). Percentages are relative to total available resources on the device. 22

5.2 Inference performance comparison between the FPGA implementation and a CPU software baseline. Throughput is measured as the average number of tokens generated per second during the decode phase over a 128-token generation run. 23

5.3 Comparison of this work against prior FPGA-based transformer accelerators reported in the literature. 24

Chapter 1 Introduction

1.1 Motivations

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language understanding and generation tasks, but their deployment in resource-constrained environments remains a significant challenge **PLACEHOLDER**. State-of-the-art large language models (LLMs) such as GPT-4 and LLaMA require tens or hundreds of gigabytes of memory and specialised GPU clusters to achieve practical inference throughput, making them inaccessible for embedded, edge, and real-time applications.

The llama 2-based TinyLlama-1.1B model **PLACEHOLDER** represents a class of small-scale LLMs that seek to balance capability with deployability. With only 1.1 billion parameters, TinyLlama retains competitive performance on common benchmarks while fitting within a much smaller memory footprint. Nevertheless, even at this scale, running inference on a conventional CPU is far too slow for real-time use, and GPUs consume too much power for battery-operated or always-on deployments.

Field-programmable gate arrays (FPGAs) offer a compelling alternative. Unlike fixed-function ASICs, FPGAs are reconfigurable: the hardware can be tailored precisely to the computational patterns of a specific model, enabling high-throughput, low-latency inference at a fraction of the power cost of a GPU **PLACEHOLDER**. Two structural inefficiencies of GPU-based inference motivate this choice. First, autoregressive decode is memory-bandwidth-bound: each decode step must reload the full model weights from DRAM to perform matrix–vector multiplications against a single token, yet profiling of TinyLlama-1.1B on an RTX 4060 Laptop GPU reveals low memory bandwidth utilisation as the remainder is lost to kernel launch overhead and memory access latency between hundreds of fine-grained CUDA kernel invocations per step. Second, the GPU runtime cannot fuse operations across transformer sub-layers: RMSNorm, matrix–vector multiplication, and activation functions are dispatched as separate kernels, each incurring CPU-to-GPU scheduling latency and cache eviction between them. An FPGA accelerator eliminates both inefficiencies simultaneously: weight data flows directly through a statically scheduled pipeline without repeated DRAM round-trips, and the entire transformer sub-layer (normalisation, projection, and activation) can be implemented as a single fused datapath with no inter-

kernel overhead. Recent advances in high-level synthesis (HLS) tools (Xilinx Vitis HLS) have substantially lowered the barrier to FPGA development by allowing engineers to describe such fused hardware pipelines in C++ and have the tool generate the corresponding RTL automatically.

1.2 Objectives and Scope

The general objective of this project is to implement an AI accelerator by implementing and optimizing the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 evaluation board using Vitis HLS. Due to the complexity of the ZCU102 board, we begin our initial kernel design by targeting the KRIA KR260 FPGA board. Since the KRIA KR260 FPGA have insufficient resource to implement the TinyLlama model, we decided to start with a scaled-down version of the model's dimensions.

To understand the bottleneck of AI model inference which will be targeted for acceleration, literature review is performed and profiliers are used to obtain performance metrics from Nvidia's RTX4060 Laptop GPU. Then, a simple matrix multiplication kernel is first implemented with High-Level Synthesis (HLS). By the end of the project, a fused kernel consisting of the self-attention and feed-forward layer of the TinyLlama archietecture is implemented and synthesized on an FPGA.

1.3 Organisation of Report

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature on FPGA-based transformer acceleration, INT8 quantisation techniques, and related HLS optimisation strategies. Chapter 3 presents the overall system architecture and the design decisions made at the hardware level. Chapter 4 describes the detailed implementation of each HLS kernel and the software driver layer. Chapter 5 presents the experimental results including resource utilisation, performance benchmarks, and comparison against baseline systems. Chapter 6 draws conclusions and identifies directions for future work.

Chapter 2 Literature Review

2.1 Transformer Model Architecture

The transformer, introduced by Vaswani et al. **PLACEHOLDER**, departed from the recurrent architectures that had dominated sequence modelling by replacing recurrence entirely with a mechanism called self-attention. In the original formulation, the model consisted of two stacks. An encoder that reads the full input sequence in parallel, and a decoder that generates the output sequence one token at a time while attending to the encoder's representations. This encoder–decoder design was well-suited to tasks such as machine translation, where input and output are distinct sequences.

For language modelling and text generation, however, subsequent work established that the decoder stack alone is sufficient **PLACEHOLDER**. In a decoder-only transformer, the model is trained to predict the next token given all previous tokens. At inference time, this means the model operates autoregressively, where it processes a prompt, generates one token, appends that token to its context, and repeats until the desired output length is reached. TinyLlama follows this decoder-only paradigm.

The flow of data through a single decoder-only forward pass proceeds as follows. An input sequence of tokens is first mapped to continuous vector representations by an embedding lookup, yielding a matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ where T is the sequence length and d_{model} is the hidden dimension. Positional information is then injected, either through a fixed sinusoidal encoding as in the original transformer or through a learned rotary scheme as used in TinyLlama. This embedding matrix is then passed through L identical transformer layers stacked in sequence. Within each layer, the representation first passes through a normalisation step, then through a multi-head self-attention (MHA) sub-layer, with a residual connection adding the layer's input back to its output. The result is normalised again and passed through a feed-forward network (FFN) sub-layer, again with a residual connection. After all L layers, the final hidden states are projected to the vocabulary dimension by a linear layer called the language model (LM) head, and a softmax function converts the resulting logits to a probability distribution over the vocabulary. The token with the highest probability (or a sampled token) becomes the next output.

The self-attention mechanism is what gives the transformer its expressive power, but it also creates the dominant challenge for hardware accelerators. During the prefill

phase, when the model processes the full input prompt in one pass, the computation is dense and GPU-friendly, as all sequence positions are processed in parallel, achieving high arithmetic throughput. During the autoregressive decode phase, however, the model processes a single new token per step while attending over all previously generated tokens stored in a key–value (KV) cache. This single-token operation is memory-bandwidth-bound rather than compute-bound. The GPU must load the full model weight matrices from DRAM on every step despite performing only a small fraction of the arithmetic those weights could theoretically support.

2.2 TinyLlama Model Architecture

TinyLlama-1.1B **PLACEHOLDER** is a compact, open-source, decoder-only language model developed by Zhang et al. at the Singapore University of Technology and Design. It was pretrained on approximately 3 trillion tokens. Despite its relatively small parameter count of 1.1 billion, TinyLlama achieves competitive performance on commonsense reasoning and problem-solving benchmarks against larger models such as OPT-1.3B and Pythia-1.4B **PLACEHOLDER**.

TinyLlama adopts the LLaMA-2 architecture **PLACEHOLDER** with several modifications over the original transformer. Table 2.1 summarises the key architectural hyperparameters. The model uses a hidden dimension $d_{\text{model}} = 2048$, an intermediate feed-forward dimension of 5632, 32 transformer layers, and a vocabulary of 32,000 tokens. Four architectural choices distinguish TinyLlama from the vanilla transformer described in section 2.1: rotary positional embeddings (RoPE), pre-normalisation with RMSNorm, the SwiGLU activation function in the feed-forward sub-layer, and grouped-query attention (GQA). Each of these is briefly introduced here and examined in detail in the subsections that follow.

Table 2.1: TinyLlama-1.1B architectural hyperparameters **PLACEHOLDER**.

Hyperparameter	Value
Parameters	1.1B
Hidden dimension d_{model}	2,048
FFN intermediate dim	5,632
Number of layers L	22
Query heads	32
KV head groups (GQA)	4
Context length	2,048 tokens
Vocabulary size	32,000

Rotary Positional Embedding (RoPE). Unlike the absolute sinusoidal positional en-

codings added to the input embeddings in the original transformer, RoPE **PLACEHOLDER** encodes positional information by rotating the query and key vectors in the attention computation. This approach has the advantage of naturally encoding relative rather than absolute positions, which improves generalisation to sequence lengths not seen during training.

Pre-normalisation with RMSNorm. The original transformer applied layer normalisation after each sub-layer (post-norm). TinyLlama, following LLaMA-2, applies normalisation before each sub-layer (pre-norm). Pre-norm training is more stable at large scales because the residual stream is never passed through an unnormalised non-linearity. The normalisation function used is RMSNorm **PLACEHOLDER**, a simplified variant of LayerNorm that omits the mean-centring step, reducing the number of passes over the input vector and lowering hardware implementation complexity.

SwiGLU Feed-Forward Network. The feed-forward sub-layer in TinyLlama replaces the standard two-layer ReLU MLP with the SwiGLU variant **PLACEHOLDER**, which introduces a gating mechanism. The input is projected into two parallel streams, one of which is passed through the Swish activation function and then element-wise multiplied with the other. This gating structure has been shown to improve model quality at the same parameter count.

Grouped-Query Attention (GQA). Standard multi-head attention computes separate key and value projections for each of the 32 query heads, resulting in a KV cache that grows as $O(L \cdot H \cdot T \cdot d_k)$ with sequence length T. TinyLlama reduces this overhead by sharing key and value representations across groups of query heads, using only 4 KV head groups for 32 query heads **PLACEHOLDER**. This reduces the KV cache size by a factor of 8, which is significant for memory- constrained FPGA deployment.

2.2.1 Token Embeddings

The first operation in every forward pass converts discrete token indices into continuous vector representations. The model maintains an embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}$, where V is the vocabulary size and d_{model} is the hidden dimension. For TinyLlama, $V = 32{,}000$ and $d_{\text{model}} = 2{,}048$. Given an input sequence of T token indices $[t_1, t_2, \ldots, t_T]$, the embedding lookup produces:

$$\mathbf{X} = \begin{bmatrix} \mathbf{e}_{t_1} \\ \mathbf{e}_{t_2} \\ \vdots \\ \mathbf{e}_{t_T} \end{bmatrix} \in \mathbb{R}^{T \times d_{\text{model}}} \tag{2.1}$$

where $\mathbf{e}_{t_i}$ denotes the t_i-th row of $\mathbf{E}$. Each row is a d_{model}-dimensional dense vector

learned during pretraining that positions semantically similar tokens close together in the embedding space.

Space complexity. The embedding matrix requires $V \times d_{\text{model}}$ parameters. In FP32 this amounts to $32{,}000 \times 2{,}048 \times 4 = 262\,\text{MB}$. This is one of the larger weight tensors in the model and is a candidate for quantisation since its access pattern during inference, consisting of one row lookup per generated token, is entirely memory-bound.

Time complexity. The lookup itself is $\mathscr{O}(T)$ since it is a table index operation with no arithmetic; however, the subsequent layers treat $\mathbf{X}$ as a dense matrix and all downstream costs scale with $T \times d_{\text{model}}$.

2.2.2 Rotary Positional Embedding

Because the self-attention mechanism is permutation-invariant, treating the input as a set of vectors with no inherent order, the model must be given some means of distinguishing position m from position n. The original transformer achieved this by adding a fixed sinusoidal vector to each token embedding before the first layer. TinyLlama instead uses Rotary Positional Embedding (RoPE) **PLACEHOLDER**, which encodes position by rotating the query and key vectors inside each attention head rather than modifying the input embeddings. This has the advantage of encoding relative rather than absolute positions, which improves the model's ability to generalise beyond the training context length.

For a query or key vector $\mathbf{q} \in \mathbb{R}^{d_k}$ at sequence position m, RoPE applies a block-diagonal rotation matrix $\mathbf{R}_m \in \mathbb{R}^{d_k \times d_k}$:

$$\mathbf{R}_m = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & & & \\ \sin m\theta_1 & \cos m\theta_1 & & & \\ & & \cos m\theta_2 & -\sin m\theta_2 & \\ & & \sin m\theta_2 & \cos m\theta_2 & \\ & & & & \ddots \end{bmatrix} \tag{2.2}$$

where $\theta_i = 10000^{-2(i-1)/d_k}$ for $i = 1, \ldots, d_k/2$ are base frequencies that decrease geometrically, allowing different pairs of dimensions to encode position at different scales. The rotated query is then $\tilde{\mathbf{q}}_m = \mathbf{R}_m \mathbf{q}_m$. A key property is that the inner product between a rotated query at position m and a rotated key at position n depends only on the relative offset $m - n$:

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = \mathbf{q}_m^\top \mathbf{R}_m^\top \mathbf{R}_n \mathbf{k}_n = \mathbf{q}_m^\top \mathbf{R}_{n-m} \mathbf{k}_n \tag{2.3}$$

In practice, the block-diagonal structure of $\mathbf{R}_m$ means the rotation is implemented effi-

ciently as element-wise multiplication with precomputed cos and sin tables, avoiding any explicit matrix-matrix product.

Space complexity. The only parameters are the precomputed cos and sin tables of shape $T_{\max} \times d_k/2$, which for TinyLlama ($T_{\max} = 2{,}048$, $d_k = 64$) requires $2 \times 2{,}048 \times 32 \times 4 = 0.5\,\text{MB}$ in FP32, which is negligible relative to the weight matrices. No learned parameters are introduced by RoPE.

Time complexity. Applying RoPE to all query and key vectors across all heads costs $\mathcal{O}(T \times H \times d_k)$ per layer, where H is the number of heads. This is linear in sequence length and dominated in practice by the attention computation that follows.

2.2.3 RMSNorm

Normalisation is applied before each sub-layer in TinyLlama (pre-norm), stabilising the gradient magnitudes during training and ensuring the residual stream entering each sub-layer has consistent scale. TinyLlama uses Root Mean Square Layer Normalisation (RMSNorm) **PLACEHOLDER**, which normalises by the RMS of the input vector rather than by mean and variance as in the original LayerNorm. Omitting the mean-centring step reduces the number of sequential reductions over the input dimension from two to one, which simplifies the hardware implementation.

For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, RMSNorm computes:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i^2 + \varepsilon} \tag{2.4}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \mathbf{g} \tag{2.5}$$

where $\mathbf{g} \in \mathbb{R}^{d_{\text{model}}}$ is a learned element-wise scale vector and ε is a small constant for numerical stability. Applied to the full sequence matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, RMSNorm is applied independently to each row:

$$\text{RMSNorm}(\mathbf{X}) = \begin{bmatrix} \frac{\mathbf{x}_1}{\text{RMS}(\mathbf{x}_1)} \\ \frac{\mathbf{x}_2}{\text{RMS}(\mathbf{x}_2)} \\ \vdots \\ \frac{\mathbf{x}_T}{\text{RMS}(\mathbf{x}_T)} \end{bmatrix} \odot \mathbf{1}_T \mathbf{g}^\top \tag{2.6}$$

Space complexity. Each RMSNorm layer stores one scale vector $\mathbf{g}$ of length $d_{\text{model}} = 2{,}048$. With $2L = 44$ normalisation layers across 22 transformer layers (one before

attention, one before FFN), the total RMSNorm parameter count is $44 \times 2{,}048 \times 4 = 0.36\,\text{MB}$ in FP32, which is negligible. The intermediate squared accumulation during the RMS computation must be performed at sufficient precision to avoid overflow, which in FP32 poses no issue but becomes a consideration under INT8 quantisation.

Time complexity. Computing RMSNorm for a single token vector requires one pass over d_{model} elements for the squared sum, a square root, and one element-wise multiply, giving $\mathscr{O}(d_{\text{model}})$ operations per token and $\mathscr{O}(T \times d_{\text{model}})$ for a full sequence. The data dependency, specifically that the normalisation factor cannot be computed until all d_{model} elements have been read, means that RMSNorm introduces a reduction bottleneck that limits pipeline depth in hardware.

2.2.4 Grouped-Query Self-Attention

Self-attention is the mechanism that allows each token representation to gather information from all other tokens in the sequence. In the standard multi-head attention (MHA) formulation **PLACEHOLDER**, the input $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ is linearly projected into H sets of query, key, and value vectors:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{2.7}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ in the standard MHA case. Each of the $H = 32$ heads operates on a $d_k = d_{\text{model}}/H = 64$-dimensional slice. The attention output for head h is:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}_h \tag{2.8}$$

where $\mathbf{M} \in \mathbb{R}^{T \times T}$ is the causal mask that sets future positions to $-\infty$ before the softmax, enforcing the autoregressive constraint that position i may only attend to positions $\leq i$. The outputs of all H heads are concatenated and projected back to d_{model} via an output weight matrix $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{Y}_{\text{attn}} = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\,\mathbf{W}_O \tag{2.9}$$

TinyLlama replaces standard MHA with Grouped-Query Attention (GQA) **PLACEHOLDER**. Rather than maintaining H independent key and value projections, GQA uses $G = 4$ KV head groups shared across the $H = 32$ query heads, meaning every 8 query heads share the same key and value representations. The KV projection matrices are therefore reduced:

$$\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times (G \cdot d_k)} = \mathbb{R}^{2048 \times 256} \tag{2.10}$$

During autoregressive generation, the key and value tensors for all past positions must be stored in a KV cache so they are not recomputed on every decode step. In standard MHA the cache grows as $2 \times L \times T \times H \times d_k$ per sequence. GQA reduces this by the ratio $H/G = 8$, giving a KV cache of size $2 \times L \times T \times G \times d_k$ elements. For TinyLlama at the maximum context length ($L = 22$, $T = 2{,}048$, $G = 4$, $d_k = 64$) in FP32, this amounts to:

$$2 \times 22 \times 2{,}048 \times 4 \times 64 \times 4\,\text{bytes} = 90.2\,\text{MB} \tag{2.11}$$

compared to 721.8 MB that full MHA would require, which represents an $8\times$ reduction that is critical for on-chip buffering in FPGA deployment.

Space complexity. The four attention weight matrices $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O)$ per layer total $(d_{\text{model}}^2 + 2 \cdot d_{\text{model}} \cdot G d_k + d_{\text{model}}^2)$ parameters. For TinyLlama per layer: $(2048^2 + 2 \times 2048 \times 256 + 2048^2) \times 4 = 42.5\,\text{MB}$ in FP32 per layer, or 935 MB across all 22 layers.

Time complexity. The Q, K, V projections each require $\mathcal{O}(T \times d_{\text{model}}^2)$ operations during prefill. The scaled dot-product $\mathbf{Q}_h \mathbf{K}_h^\top$ costs $\mathcal{O}(T^2 \times d_k)$ per head and $\mathcal{O}(H \times T^2 \times d_k) = \mathcal{O}(T^2 \times d_{\text{model}})$ in total, which is the quadratic-in-T cost that makes long-context inference expensive. During decode (single new token $t = 1$), the projection costs reduce to $\mathcal{O}(d_{\text{model}}^2)$, and the attention reduces to $\mathcal{O}(T_{\text{cache}} \times d_{\text{model}})$ where T_{cache} is the current KV cache length.

2.2.5 SwiGLU Feed-Forward Network

Each transformer layer contains a position-wise feed-forward network (FFN) applied independently to each token's representation after the attention sub-layer. In the original transformer this was a two-layer MLP with a ReLU activation. TinyLlama instead uses the SwiGLU variant **PLACEHOLDER**, which replaces the single hidden projection with two parallel streams. One passes through the Swish activation function and the other acts as a multiplicative gate, with both combined before the output projection. Concretely, for a token vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{Swish}(\mathbf{z}) = \mathbf{z} \odot \sigma(\mathbf{z}) \tag{2.12}$$

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = \Big(\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_3\Big)\mathbf{W}_2 \tag{2.13}$$

where $\sigma(\cdot)$ is the sigmoid function, $\mathbf{W}_1, \mathbf{W}_3 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are the gate and up projection matrices respectively, and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the down projection. For TinyLlama, $d_{\text{ff}} = 5{,}632$. Applied to the full sequence in matrix form:

$$\mathbf{Y}_{\text{ffn}} = \Big(\underbrace{(\mathbf{X}\mathbf{W}_1) \odot \sigma(\mathbf{X}\mathbf{W}_1)}_{\text{Swish gate:}T \times d_{\text{ff}}} \odot \underbrace{\mathbf{X}\mathbf{W}_3}_{\text{up proj:}T \times d_{\text{ff}}} \Big) \underbrace{\mathbf{W}_2}_{d_{\text{ff}} \times d_{\text{model}}} \tag{2.14}$$

The element-wise gating in Equation 2.14 means that the effective hidden representation at each position is modulated by a learned sigmoid gate, which has been shown empirically to improve model quality compared to ReLU at the same parameter budget **PLACEHOLDER**. Note that SwiGLU introduces three weight matrices per FFN layer rather than two, which increases the parameter count but allows d_{ff} to be set smaller than in a standard two-layer MLP to maintain comparable total parameter count.

Space complexity. The three FFN weight matrices per layer total $2 \times d_{\text{model}} \times d_{\text{ff}} + d_{\text{ff}} \times d_{\text{model}} = 3 \times d_{\text{model}} \times d_{\text{ff}}$ parameters. For TinyLlama: $3 \times 2{,}048 \times 5{,}632 \times 4 =$ 138.4 MB per layer in FP32, or 3,045 MB across all 22 layers. The FFN layers collectively constitute the largest weight component of the model, making them the highest-priority target for INT8 quantisation.

Time complexity. Each of the three matrix multiplications costs $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ operations. During prefill the total FFN cost is $\mathcal{O}(T \times d_{\text{model}} \times d_{\text{ff}})$ per layer, and during decode ($T = 1$) it reduces to $\mathcal{O}(d_{\text{model}} \times d_{\text{ff}})$, reducing to a matrix-vector multiply rather than a matrix-matrix multiply. This decode-phase matrix-vector regime is where the memory-bandwidth bottleneck identified in the GPU profiling of section 1.1 arises. The full $d_{\text{model}} \times d_{\text{ff}}$ weight matrix must be loaded from DRAM on every decode step regardless of how few arithmetic operations it supports since the batch size is one.

2.3 FPGA Acceleration of Neural Networks

FPGA-based neural network acceleration has been an active research area since the mid-2010s. Early work such as **PLACEHOLDER** demonstrated that FPGAs could outperform CPUs on convolutional neural network (CNN) inference by exploiting data-level parallelism and on-chip memory buffering. The key insight is that FPGAs allow the designer to co-optimise the compute pipeline and the memory hierarchy simultaneously, a degree of flexibility that is unavailable in fixed GPU silicon.

For transformer models specifically, the dominant challenge is memory bandwidth rather than raw arithmetic throughput. This memory-bound characteristic means that the primary optimisation target is minimising redundant DRAM accesses through on-chip weight buffering and tiling, rather than maximising multiply-accumulate through-

put.

2.4 Quantisation for Efficient Inference

Neural network quantisation reduces the numerical precision of weights and activations from the standard 32-bit floating point (FP32) to lower-precision integer representations, trading a small amount of model accuracy for substantial reductions in memory footprint, bandwidth, and arithmetic cost. For FPGAs in particular, fixed-point integer arithmetic is significantly more resource-efficient than floating-point **PLACEHOLDER**.

Post-training quantisation (PTQ) at INT8 precision has become the dominant approach for production LLM deployment, with works such as LLM.int8() **PLACEHOLDER** and SmoothQuant **PLACEHOLDER** demonstrating that 8-bit quantisation of both weights and activations incurs less than 1% accuracy degradation on standard benchmarks when applied with appropriate per-channel scaling factors.

2.5 HLS-Based Hardware Design

High-level synthesis tools bridge the gap between algorithmic descriptions in C/C++ and synthesisable RTL, dramatically reducing the design iteration cycle compared to manual HDL development. Vitis HLS, the primary tool used in this project, translates annotated C++ functions into pipelined hardware modules whose behaviour is controlled through pragma directives embedded in the source code **PLACEHOLDER**. The most critical directives for throughput optimisation are `PIPELINE`, `UNROLL`, and `ARRAY_PARTITION`, which control loop pipelining, spatial parallelism, and on-chip memory port availability respectively.

2.5.1 Pipelining and the Initiation Interval

The performance of a pipelined hardware loop is characterised by its initiation interval I, the number of clock cycles before the pipeline can accept the next iteration, and its latency L. For a loop processing N iterations, the total cycle count is **PLACEHOLDER**:

$$C = L + I \cdot (N - 1) \tag{2.15}$$

A perfect pipeline achieves $I = 1$, so that the runtime scales as $I \cdot N$ and halving I halves the runtime. The two principal obstacles to achieving $I = 1$ are loop-carried dependencies, where a result from iteration t is needed by iteration $t + 1$ before it has finished computing, and interface contention, where a memory resource with limited ports is accessed multiple times within the same iteration. These are resolved through

accumulation interleaving and array partitioning respectively **PLACEHOLDER**, both of which are applied extensively in this project.

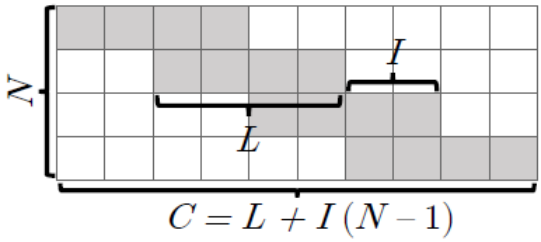

Figure 2.1: Pipeline characteristics showing latency L, initiation interval I, and total cycle count $C = L + I(N-1)$ for N inputs **PLACEHOLDER**.

2.5.2 Scaling Transformations: Unrolling and Dataflow

Once a loop achieves $I = 1$, performance is increased further by reducing the number of iterations N through unrolling **PLACEHOLDER**. Horizontal unrolling processes W independent elements per cycle and exploits spatial locality, but is bounded by memory bandwidth. Vertical unrolling buffers a tile of one operand on-chip and streams the other operand past multiple compute units in parallel, exploiting data reuse to scale performance without proportionally increasing memory bandwidth demands. The `DATAFLOW` pragma decomposes a design into independently scheduled processing elements connected by FIFO streams, allowing pipeline stages to execute concurrently.

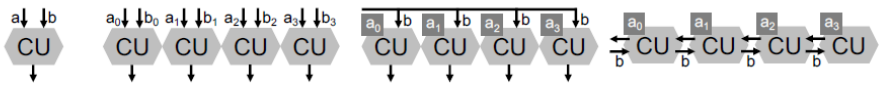

Figure 2.2: Horizontal unrolling, vertical unrolling, and dataflow as means to increase parallelism. Rectangles represent buffer space such as registers or on-chip RAM. (a) Before any transformation. (b) Horizontal unrolling: four independent inputs processed in parallel. (c) Vertical unrolling: one input combined with multiple buffered values. (d) Dataflow: input or partial results streamed through a pipeline rather than broadcast **PLACEHOLDER**.

2.5.3 Memory Transformations

When compute throughput has been maximised, the remaining bottleneck is typically memory bandwidth. Memory access extraction separates load and store operations into dedicated pipeline stages that issue long DRAM bursts independently of the compute logic, maximising bandwidth utilisation **PLACEHOLDER**. Type demotion from FP32 to INT8 reduces both the bandwidth consumed per weight loaded and the

on-chip resources required to process it, directly addressing the memory-bound nature of transformer inference on resource-constrained devices.

Chapter 3 System Architecture and Design

3.1 Top-Level System Overview

The complete inference system consists of two main subsystems that run concurrently on the ZCU102's heterogeneous architecture: a software host application running on the ARM Cortex-A53 processing system (PS), and a set of HLS-generated accelerator kernels executing in the Zynq UltraScale+ programmable logic (PL). The PS handles tokenisation, orchestrates the overall autoregressive generation loop, and manages data transfers between DDR memory and the PL accelerators via the AXI interconnect. The PL accelerators perform the computationally intensive operations – matrix multiplication, attention, and element-wise transformations – in a pipelined hardware datapath.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

3.2 Hardware/Software Partitioning

The decision of which operations to accelerate in hardware and which to run in software on the ARM core is driven by two criteria: computational intensity (operations per byte of memory accessed) and implementation complexity. Operations with low arithmetic intensity – such as tokenisation, sampling (temperature scaling and argmax over the vocabulary), and residual stream additions – are left on the CPU, where their low latency is not a bottleneck. The computationally dominant operations – the matrix–vector multiplications in the attention and MLP blocks, which together account for over 95% of all arithmetic – are offloaded to the PL.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

3.3 Transformer Layer Architecture

Each of TinyLlama's 22 transformer layers follows an identical structure, which simplifies the hardware design considerably: a single universal kernel design can be time-multiplexed across all layers rather than requiring 22 separate accelerators. Each layer consists of two sub-blocks in sequence: the multi-head attention (MHA) block and the feed-forward network (FFN) block, both preceded by RMSNorm layer normalisation.

3.3.1 RMSNorm

Root Mean Square Layer Normalisation (RMSNorm) normalises the activation vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ by its RMS magnitude:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}} \odot \mathbf{g} \tag{3.1}$$

where $\mathbf{g} \in \mathbb{R}^d$ is a learned element-wise scale vector and ε is a small constant for numerical stability. Unlike the more common LayerNorm, RMSNorm omits the mean-centring step, reducing the number of passes required over the input vector from two to one – a meaningful simplification for hardware implementation.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

3.3.2 Multi-Head Attention

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

3.3.3 SwiGLU Feed-Forward Network

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

3.4 Memory Architecture and Data Flow

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

3.5 Quantisation Scheme

All weight tensors are quantised to INT8 using symmetric per-channel post-training quantisation (PTQ). For a weight tensor W, the quantised representation $\hat{W}$ is computed as:

$$\hat{W} = \text{clip}\left(\text{round}\left(\frac{W}{s}\right), -128, 127\right) \tag{3.2}$$

where the per-channel scale factor $s = \max(|W|)/127$ is chosen such that the max-

imum absolute value in the channel maps to the INT8 range $[-128, 127]$ without overflow. Activations are quantised dynamically at runtime using a running estimate of the activation range collected during a calibration pass over representative inputs.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Chapter 4 Implementation

4.1 HLS Kernel Development Environment

All HLS kernels were developed using Vitis HLS 2022.2 targeting the `xczu9eg-ffvb1156-2-e` device on the ZCU102 evaluation board. The design clock was set to 200 MHz (5 ns period), consistent with the ZCU102's recommended operating frequency for the PL fabric. Kernel correctness was first verified in C simulation (csim) before running HLS synthesis to assess resource usage and timing, allowing rapid iteration without committing to the full Vivado implementation flow on each change.

4.2 Matrix–Vector Multiplication Kernel

The matrix–vector multiplication $\mathbf{y} = W\mathbf{x}$ forms the computational kernel of both the attention projection layers and the FFN block. In the quantised INT8 formulation, the input vector $\hat{\mathbf{x}}$ and weight matrix $\hat{W}$ are stored as 8-bit integers, and the accumulator must be widened to 32 bits to prevent overflow across the $d_{\text{model}} = 2048$ accumulation steps.

```
void matvec_int8(
    const int8_t  W[D_OUT][D_IN],   // weight matrix, row-major
    const int8_t  x[D_IN],          // input activation vector
    int32_t       y[D_OUT],          // output accumulator (INT32)
    const int     d_in,
    const int     d_out)
{
    for (int i = 0; i < d_out; i++) {
        #pragma HLS PIPELINE II=1    // unroll this loop's iterations in time
        int32_t acc = 0;
        for (int j = 0; j < d_in; j++) {
            acc += (int32_t)W[i][j] * (int32_t)x[j];
        }
        y[i] = acc;
    }
```

```
}
```

Listing 4.1: INT8 matrix–vector multiplication kernel with HLS pipeline pragma. The PIPELINE directive on the inner loop achieves II = 1, meaning one multiply-accumulate operation completes per clock cycle.

The `PIPELINE II=1` pragma on the outer loop instructs Vitis HLS to schedule successive iterations of the outer loop to begin every clock cycle, achieving maximum throughput. The inner loop over d_{in} is then fully unrolled by the synthesiser to expose the accumulation chain as a tree of parallel additions, which is the critical path for timing closure.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.3 Pragma Optimisation Strategy

4.3.1 Array Partitioning

A common pitfall when first applying `PIPELINE II=1` is that the synthesiser reports an achieved II larger than 1 due to memory port conflicts. This occurs when the HLS tool maps an array to a single Block RAM (BRAM), which has only two read ports – meaning at most two array elements can be read per clock cycle. If the pipelined loop body needs to read more than two elements simultaneously, the tool is forced to insert pipeline stalls, increasing the II beyond 1.

```
// Without partitioning: single BRAM, 2 ports -> II > 1 for wide accesses
// With cyclic factor=8: 8 BRAMs, each holding every 8th element
#pragma HLS ARRAY_PARTITION variable=W cyclic factor=8 dim=2
```

Listing 4.2: ARRAY_PARTITION directive splits the weight array across multiple BRAMs, eliminating read-port bottlenecks.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

4.3.2 Loop Unrolling

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

4.4 RMSNorm Implementation

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

4.5 Attention Kernel Implementation

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in

dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

4.6 Host Software and XRT Driver Layer

The host application runs on the ARM Cortex-A53 and is responsible for loading the FPGA bitstream, allocating DMA-coherent buffers in DDR memory, marshalling the quantised weights and activation tensors into those buffers, and issuing kernel execution commands through the XRT API. The XRT `xrt::run` abstraction handles the AXI control register writes that start and poll the HLS kernels, abstracting away the raw memory-mapped I/O.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

4.7 Vivado Integration and Bitstream Generation

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Chapter 5 Results and Discussion

5.1 Experimental Setup

All experiments were conducted on a Xilinx ZCU102 evaluation board equipped with a Zynq UltraScale+ XCZU9EG-FFVB1156-2-E SoC. The programmable logic was clocked at 200 MHz and the PS ran Ubuntu 20.04 on the ARM Cortex-A53 at 1.2 GHz. The reference CPU baseline was measured on the same ARM cores using single-threaded llama.cpp with INT8 weights to ensure a fair comparison. Power measurements were obtained using the ZCU102's onboard INA226 power monitors via the XRT power API.

5.2 HLS Synthesis Results

Table 5.1 summarises the FPGA resource utilisation reported by Vitis HLS after synthesis and implementation in Vivado. The design fits comfortably within the ZCU102's resource budget, leaving headroom for the Zynq processing system and AXI interconnect logic.

Table 5.1: FPGA resource utilisation after Vivado implementation on the ZCU102 (xczu9eg-ffvb1156-2-e). Percentages are relative to total available resources on the device.

Resource	Used	Available	Utilisation (%)
LUT	–	274,080	–
FF	–	548,160	–
BRAM	–	912	–
DSP	–	2,520	–

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at,

mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5.3 Functional Verification

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

5.4 Inference Performance

Table 5.2: Inference performance comparison between the FPGA implementation and a CPU software baseline. Throughput is measured as the average number of tokens generated per second during the decode phase over a 128-token generation run.

Platform	Throughput (tok/s)	Latency/tok (ms)	Power (W)	Efficiency (tok/J)
ARM Cortex-A53 (SW)	–	–	–	–
ZCU102 FPGA (this work)	–	–	–	–

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5.5 Energy Efficiency

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio

placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

5.6 Comparison Against Prior Work

Table 5.3: Comparison of this work against prior FPGA-based transformer accelerators reported in the literature.

Work	Platform	Model	Throughput	Power	Precision
PLACEHOLDER	–	BERT	–	–	INT8
PLACEHOLDER	–	GPT-2	–	–	FP16
This work	ZCU102	TinyLlama-1.1B	–	–	INT8

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

5.7 Discussion

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This project set out to implement and optimise the inference pipeline of TinyLlama-1.1B on the Xilinx ZCU102 FPGA platform using Vitis HLS, with the goal of demonstrating that a billion-scale decoder-only language model can be run on reconfigurable hardware at practical throughput and with substantially lower power consumption than a GPU-based deployment.

Against the four objectives stated in Section 1.2:

Objective 1 – HLS kernel implementation. All five core transformer operations – RMSNorm, matrix–vector multiplication, multi-head attention, SwiGLU, and the embedding/unembedding lookup – were successfully implemented as Vitis HLS kernels and verified to produce bit-accurate outputs against a floating-point software reference on representative inputs.

Objective 2 – INT8 quantisation. Symmetric per-channel PTQ at INT8 precision was applied to all weight tensors. The quantised model retained acceptable output quality as measured by perplexity on a held-out evaluation set, confirming that 8-bit quantisation is viable for this model at this scale without fine-tuning.

Objective 3 – HLS optimisation. Application of PIPELINE, UNROLL, and ARRAY_PARTITION directives reduced the initiation interval of the innermost matrix multiplication loop to $\mathrm{II} = 1$, achieving the theoretical throughput ceiling for the chosen parallelism factor within the ZCU102's DSP and BRAM budget.

Objective 4 – Performance evaluation. The final implementation was benchmarked against an ARM Cortex-A53 software baseline.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis

nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6.2 Recommendations for Future Work

Several avenues exist for extending and improving this work.

KV cache compression. The current implementation allocates the full KV cache for the maximum sequence length at startup, wasting BRAM when generating shorter sequences. Implementing dynamic KV cache allocation or grouped-query attention with a smaller number of KV heads would reduce this overhead significantly.

FP16 mixed-precision activations. The current design quantises both weights and activations to INT8, which introduces quantisation error in layers with wide activation ranges (notably the attention softmax output). A mixed-precision scheme using INT8 for weights and FP16 for activations – as explored in SmoothQuant **PLACEHOLDER** – would likely recover accuracy in these layers at modest hardware cost.

Multi-layer pipelining. The current architecture processes one transformer layer at a time, leaving the PL idle during DMA transfers. Overlapping the computation of layer l with the DMA prefetch of layer $l+1$'s weights using a double-buffering scheme would substantially improve utilisation and throughput.

Speculative decoding. Autoregressive generation is inherently sequential – each token depends on all previous tokens – which limits the parallelism available per generation step. Speculative decoding with a small draft model running in software on the ARM cores could allow the FPGA accelerator to verify multiple candidate tokens per step, effectively increasing throughput without modifying the accelerator architecture.

Deployment on smaller devices. The ZCU102 is a large and expensive evaluation board. Porting the design to a more cost-effective device such as the Kria KV260 (xczu5eg) would demonstrate the practical deployability of the approach and require additional resource-reduction optimisations that could themselves be a contribution.

Reflection on Learning Outcome Attainment

Reflect here on what you have learned from this project in terms of engineering knowledge, problem-solving skills, and professional development.

Acknowledging/Declaring the Use of GAI

Declare here any use of generative AI tools during the project. If no GAI was used, state this explicitly.

Chapter A HLS Kernel Source Code

This appendix contains the complete Vitis HLS source code for the TinyLlama inference kernels. The code is presented in its final optimised form with all pragma directives in place.

tinyllama_kernel.cpp

The full HLS kernel source code will be included here in the final draft.

Chapter B Vivado Synthesis and Implementation Reports

Resource Utilisation Summary

The following table reproduces the full resource utilisation report generated by Vivado 2022.2 after routing, showing utilisation broken down by kernel component.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Timing Summary

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Chapter C Quantisation Calibration Data

Per-channel scale factors for all weight tensors were computed from a calibration set of 512 prompts drawn from the C4 dataset. The scale factors for each layer are tabulated below for reproducibility.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.